BIBLIOUSFX v1.0

Plan de Aseguramiento de Calidad de Software (SQAP)

Versión del documento: 1.0 · Estado: Aprobado por el equipo

Equipo M-07 · Facultad de Ciencia y Tecnología · Gestión II/2026

Conforme a IEEE Std 730-2002 · Modelo de calidad ISO/IEC 25010:2011

1. Propósito

Este plan cubre el desarrollo de BIBLIOUSFX v1.0, aplicación web de gestión de biblioteca de la Facultad de Ciencia y Tecnología. Abarca las fases de requerimientos, diseño, construcción y pruebas de los casos de uso **Buscar libro**, **Registrar préstamo**, **Registrar devolución** y **Calcular multa**, durante la Gestión II/2026.

El sistema se clasifica en el **nivel de integridad 2** (esquema de IEEE 1012): su falla causa inconveniencia operativa a estudiantes y bibliotecarios, pero no pérdida de vidas ni daño económico mayor. Los módulos **Préstamos** y **Multas** se consideran críticos dentro del sistema porque afectan registros académicos y montos cobrados.

2. Documentos de referencia

Código	Documento	Versión
SRS	Especificación de requerimientos de BIBLIOUSFX (extracto de RNF en el Anexo)	1.2
SAD	Documento de arquitectura: Spring Boot 3.3 + PostgreSQL 16 + React 18	1.0
IEEE 730	IEEE Std 730-2002, Software Quality Assurance Plans	2002
ISO 25010	ISO/IEC 25010:2011, modelo de calidad del producto	2011
IEEE 1028	IEEE Std 1028, Software Reviews and Audits	2008
ESTILO	Google Java Style Guide; Airbnb JavaScript Style Guide	—

3. Gestión

El equipo M-07 tiene cuatro integrantes con roles explícitos. Cada integrante tiene más de un rol por el tamaño del equipo.

Rol	Responsabilidad principal	Integrante
Líder de proyecto	Coordinación, cronograma, comunicación con el docente	Ana Quispe
Arquitecto	Decisiones de arquitectura (ADR), revisión de diseño	Bruno Mamani
Responsable SQA	Plan SQA, métricas, auditorías, aprobación de pull requests de los módulos críticos. No programa los módulos críticos.	Carla Flores
Desarrollador backend	API REST y pruebas JUnit de los módulos Préstamos y Multas	Bruno Mamani y Ana Quispe
Desarrollador backend (búsqueda)	API y pruebas JUnit de Buscar libro (no crítico)	Carla Flores
Desarrollador frontend y tester	Interfaz React, pruebas Cypress y JMeter	Diego Choque

Regla de independencia: la Responsable SQA no programa los módulos críticos (Préstamos y Multas), que desarrollan Bruno Mamani y Ana Quispe; por eso ella aprueba sus pull requests. Los pull requests de Carla Flores (Buscar libro) los revisa otro integrante (Sección 6).

4. Documentación

Documento	Responsable	Fecha	Tipo de revisión
SRS 1.2	Ana Quispe	Semana 1	Revisión con el cliente + inspección Fagan
SAD 1.0	Bruno Mamani	Semana 2	Inspección Fagan
Manual de usuario	Diego Choque	Semana 7	Revisión técnica
Informe de pruebas	Diego Choque	Cada release	Auditoría SQA

5. Estándares, prácticas, convenciones y métricas

Java sigue Google Java Style Guide y JavaScript sigue Airbnb JavaScript Style Guide, verificados por SonarQube. Las clases de prueba terminan en `Test` y sus métodos describen el comportamiento esperado (`multaSeCalculaPorDiaDeRetraso`). Endpoints en plural y minúsculas (`/api/libros`, `/api/prestamos`).

Característica	Métrica	Instrumento, condiciones y momento	Objetivo	RNF
Adecuación funcional	% de casos de prueba de Multas que pasan	JUnit 5, en cada build del pipeline	100 %	RNF-06
Eficiencia de desempeño	Percentil 95 del tiempo de respuesta de la búsqueda	JMeter (Práctica 5), 100 usuarios concurrentes, 10 minutos, ambiente de staging; etapa Rendimiento del pipeline en cada release candidate	≤ 800 ms	RNF-01
Usabilidad	Tasa de éxito de la tarea "registrar un préstamo" sin ayuda; puntaje SUS	Prueba con 5 estudiantes que nunca usaron el sistema, al cierre del sprint 3; cuestionario SUS de 10 ítems	≥ 90 % de éxito; SUS promedio ≥ 68	RNF-03
Fiabilidad	Disponibilidad mensual en horario de servicio	Monitor externo cada 5 minutos, informe mensual	≥ 99,0 %	RNF-02
Seguridad	Vulnerabilidades altas o críticas	SonarQube y Dependabot en cada pull request	0	RNF-04
Mantenibilidad	Cobertura de líneas de Préstamos y Multas	JaCoCo en cada build	≥ 80 %	RNF-05
Mantenibilidad	Deuda técnica	SonarQube en cada build	< 5 días-persona	RNF-05

6. Revisiones de software

Revisión	Artefacto	Cuándo	Quién revisa	Criterio de aceptación
Revisión de requerimientos	SRS	Semana 1	Cliente, Arquitecto, SQA	Cada RNF tiene criterio verificable
Inspección Fagan	SAD	Semana 2	Moderadora: Carla Flores; inspectores: Ana y Diego	0 defectos mayores abiertos
Revisión de código	Pull request	Cada PR	Responsable SQA (módulos críticos); otro integrante (resto)	Lista de verificación de PR completa; Quality Gate en verde

Revisión	Artefacto	Cuándo	Quién revisa	Criterio de aceptación
Auditoría de cierre	Cumplimiento del SQAP	Semana 8	Responsable SQA y docente	Siete indicadores de la regla cumplidos

7. Pruebas

Nivel	Técnica y herramienta	Responsable	Criterio de salida
Unitario	TDD obligatorio en Préstamos y Multas; JUnit 5 + JaCoCo	Desarrolladores	100 % en verde; cobertura de líneas ≥ 80 % en Préstamos y Multas, ≥ 60 % global
Integración	Pruebas de API con Testcontainers (PostgreSQL)	Desarrolladores	0 fallos
Sistema	E2E con Cypress; carga con JMeter	Diego Choque	Escenarios críticos en verde; objetivos de la Sección 5 cumplidos
Aceptación	Escenarios con la bibliotecaria; prueba de usabilidad	Ana Quispe	0 defectos S1 o S2 abiertos; acta firmada

8. Reporte de problemas y acción correctiva

Los defectos se registran como issues de GitHub con la plantilla del equipo. Estados: nuevo → asignado → en progreso → resuelto → verificado → cerrado; desde verificado o cerrado puede pasar a reabierto. La severidad la fija la Responsable SQA; la prioridad, la líder de proyecto.

Severidad	Definición	Respuesta	Resolución
Crítica (S1)	Sistema inusable o pérdida de datos	2 horas	1 día hábil
Alta (S2)	Funcionalidad principal afectada sin alternativa	1 día hábil	3 días hábiles
Media (S3)	Funcionalidad secundaria afectada o con alternativa	2 días hábiles	1 semana
Baja (S4)	Cosmético o mejora menor	3 días hábiles	Siguiente sprint

9. Herramientas, técnicas y metodologías

Repositorio en GitHub; integración continua con Jenkins (`Jenkinsfile` en la raíz del repositorio); análisis estático con SonarQube; JUnit 5, JaCoCo, Testcontainers, Cypress y JMeter. El backend usa 21 dependencias directas de Maven y el frontend 22 de npm.

Configuración del Quality Gate “BIBLIOUSFX” en SonarQube:

```
Coverage on New Code is less than 80.0       -> falla
Duplicated Lines on New Code > 3.0 %         -> falla
Security Rating on New Code worse than A     -> falla
Maintainability Rating on New Code worse than A -> falla
```

Regla de JaCoCo en el `pom.xml` (paquetes `prestamos` y `multas`), ejecutada en la etapa Cobertura del `Jenkinsfile`:

```
<limit>
    <counter>LINE</counter>
    <value>COVEREDRATIO</value>
    <minimum>0.80</minimum>
</limit>
```

10. Control de medios

GitHub Flow: la rama `main` está protegida (sin push directo, un pull request aprobado y el build de Jenkins en verde para hacer merge). El repositorio se respalda cada semana con `git bundle` en el Drive de la facultad, con retención de 3 meses; la restauración se prueba una vez por mes. La base de datos de staging se respalda cada noche con `pg_dump`. Solo la líder y el arquitecto tienen permiso de administración en GitHub y Jenkins.

11. Control de proveedores

Criterios de selección: toda dependencia nueva debe tener una licencia compatible con el uso académico (MIT, Apache 2.0, EPL 2.0 o BSD), al menos un release en los últimos 12 meses y una comunidad activa. Entra al proyecto por un pull request que la registra en la hoja Componentes de terceros de matrices-sqap.xlsx (nombre, versión, licencia, justificación, responsable y fecha de la última revisión de seguridad).

Inventario: las 21 dependencias directas de Maven del backend y las 22 de npm del frontend (Sección 9) están registradas en esa hoja y se revisan al agregar una dependencia y una vez por mes.

Análisis de vulnerabilidades: Dependabot en cada pull request, con el mismo umbral de la Sección 5 (0 vulnerabilidades altas o críticas). Revisión de licencias: en la revisión mensual, el Arquitecto verifica que ninguna dependencia haya cambiado a una licencia incompatible. Responsable: Bruno Mamani (Arquitecto).

12. Registros: recolección, mantenimiento y retención

Evidencia	Ubicación	Retención
Actas de revisión e inspección	`docs/actas/` en el repositorio	Toda la vida del proyecto
Informes de pruebas JUnit y JaCoCo	Artefactos del build en Jenkins	30 builds
Dashboards de JMeter	Artefactos del build de cada release	Toda la vida del proyecto
Informes de SonarQube	Servidor SonarQube del equipo	6 meses
Issues de defectos	GitHub Issues	Toda la vida del proyecto

13. Capacitación

Competencia	Quién	Sesión
Testcontainers	Ana, Bruno	Taller interno de 2 horas, semana 2 (dicta Bruno)
Inspecciones Fagan	Todo el equipo	Lectura de IEEE 1028 y simulacro con el SRS, semana 1
JMeter en el pipeline	Diego	Repaso de las Prácticas 5 y 6, semana 3

14. Gestión de riesgos

Escala de 1 (bajo) a 3 (alto). Exposición = probabilidad × impacto. Nivel: 1–2 bajo, 3–4 medio, 6–9 alto.

Riesgo	P	I	Exposición	Mitigación	Resp.
Rotación o abandono de un integrante	3	3	9 — Alto	Pair programming en Préstamos y Multas; cada módulo documentado en el wiki del repositorio antes de cerrar la historia; revisión quincenal de la carga de trabajo en la planificación.	Ana
Requerimientos ambiguos de la bibliotecaria	3	2	6 — Alto	Revisión del SRS con la cliente en la semana 1; prototipo navegable validado antes del sprint 2.	Bruno

Riesgo	P	I	Exposición	Mitigación	Resp.
Deuda técnica por presión de cronograma	2	3	6 — Alto	Quality Gate bloqueante en el pipeline; 10 % de cada sprint para refactorizar.	Carla
Falla del servidor Jenkins	1	2	2 — Bajo	Configuración en Git (`Jenkinsfile`, docker-compose); reinstalación probada en la semana 2.	Carla
Vulnerabilidad crítica en una dependencia	2	3	6 — Alto	Dependabot en cada PR; actualización mensual de dependencias.	Bruno
Datos de prueba con información personal real	1	3	3 — Medio	Datos sintéticos generados con un script versionado; prohibido copiar la base de producción.	Carla

15. Glosario

Término	Definición
Préstamo	Registro de la entrega de un ejemplar a un usuario, con fecha de devolución prevista.
Multa	Monto por día de retraso en la devolución de un ejemplar.
p95	Valor por debajo del cual está el 95 % de los tiempos de respuesta medidos.
Quality Gate	Conjunto de condiciones de SonarQube que un cambio debe cumplir para aceptarse.
SUS	System Usability Scale: cuestionario de 10 ítems con puntaje de 0 a 100.

16. Procedimiento de cambio e historia del SQAP

Cualquier integrante puede proponer un cambio mediante un pull request sobre `docs/sqap.docx`. Lo aprueba la Responsable SQA y, si cambia un umbral, también la líder de proyecto.

Versión	Fecha	Cambio	Autor	Revisor
0.1	04/08/2026	Versión inicial: Secciones 1 a 4	Ana Quispe	Bruno Mamani

Versión	Fecha	Cambio	Autor	Revisor
0.5	18/08/2026	Métricas, pruebas y herramientas	Carla Flores	Diego Choque
0.9	01/09/2026	Riesgos, glosario; aprobado por el equipo	Ana Quispe	Todo el equipo
0.9.1	04/10/2026	Sección 5: métrica de usabilidad medible y trazable a RNF-03 (tasa de éxito con 5 usuarios nuevos y SUS).	Galvan	Lujan
0.9.2	04/10/2026	Sección 3: la Responsable SQA deja de programar Préstamos y Multas; los desarrollan Bruno Mamani y Ana Quispe.	Lujan	Galvan
0.9.3	04/10/2026	Sección 5: el p95 de la búsqueda se mide con JMeter, 100 usuarios, 10 min, staging, en cada release candidate (RNF-01).	Galvan	Lujan
0.9.4	04/10/2026	Sección 9: Quality Gate y regla de JaCoCo de 50 % a 80 %, igual que las Secciones 5 y 7.	Lujan	Galvan
0.9.5	04/10/2026	Nueva Sección 11, Control de proveedores (21 + 22 dependencias); las Secciones 11 a 15 pasan a ser 12 a 16.	Galvan	Lujan
0.9.6	04/10/2026	Sección 14: riesgo de rotación con exposición 3 × 3 = 9 (Alto) y mitigación accionable.	Lujan	Galvan
1.0	04/10/2026	Plan corregido tras la auditoría: cumple los siete indicadores de la regla del plan verificable, sin cambiar el alcance.	Galvan	Lujan

Anexo — Extracto de requerimientos no funcionales (SRS 1.2)

Código	Requerimiento
RNF-01	La búsqueda de libros debe responder en menos de 800 ms para el 95 % de las consultas con 100 usuarios concurrentes (horario pico de exámenes).
RNF-02	El sistema debe estar disponible el 99 % del horario de servicio (lunes a sábado, 7:00 a 22:00).

Código	Requerimiento
RNF-03	Un estudiante que nunca usó el sistema debe poder registrar un préstamo sin ayuda.
RNF-04	Los datos personales de los usuarios deben protegerse: tránsito cifrado y ninguna vulnerabilidad conocida alta o crítica.
RNF-05	El código debe poder ser mantenido por el equipo de la siguiente gestión.
RNF-06	El cálculo de multas debe ser exacto según el reglamento de la biblioteca (Bs 2 por día de retraso, tope de Bs 60).